AMD Matrix Cores

Gina Sitaraman

AMD @ HLRS
April 22-25th, 2024

AMD
together we advance_

Authors and Contributors

- Ossian O'Reilly, AMD
- Noel Chalmers, AMD
- Damon McDougall, AMD
- Rene Van Oostrum, AMD
- Nicholas Malaya, AMD
- Joseph Greathouse, AMD

Agenda

- What are Matrix Cores and why are they useful?
- How do we leverage Matrix Cores?
- Example of matrix multiplication with outer products
- Examples using compiler intrinsics

What are Matrix Cores?

- Special purpose hardware that performs general matrix multiplications (GEMM)s
- Accelerate matrix multiplications with a new family of Matrix Fused Multiply Add instructions (MFMA)
- Support mixed precision
- Achieve more FLOPS/cycle when compared to vector units

Where are Matrix Cores used?

- GEMMs used in scientific computing with FP64 data
- Seen mostly in AMD's rocBLAS library
- Also seen in machine learning applications due to their mixed precision support
  - Present in ML frameworks, PyTorch and TensorFlow
  - Matrix multiplication in Neural network architectures

Matrix Cores in AMD Instinct™ MI100

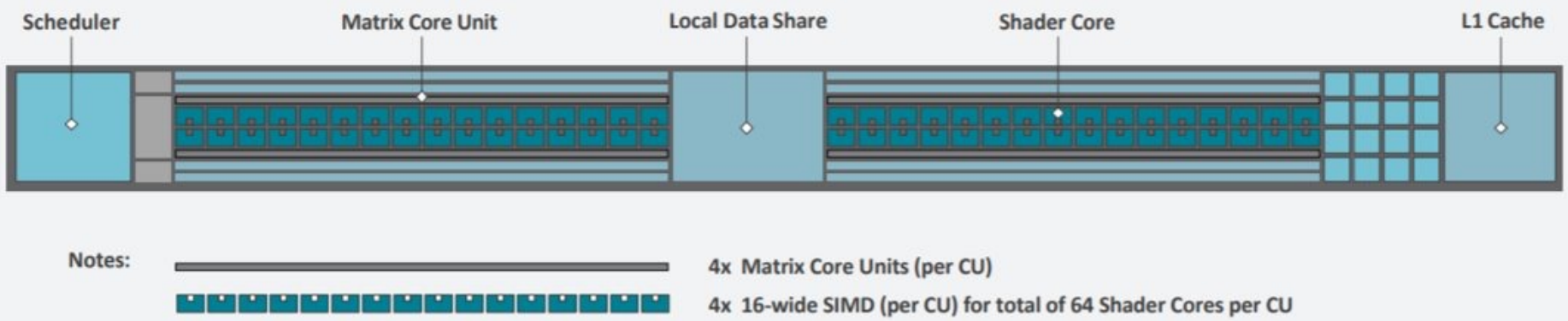


Figure 3 – Conceptional Block Diagram of an Enhanced Compute Unit (CU) with SIMD view of the AMD CDNA™ 2 architecture

- Matrix cores introduced in AMD's CDNA™ architecture (MI100)
- Only FP32 / BF16 / FP16 / Int8 Matrix Fused Multiply Add (MFMA) operations

https://www.amd.com/system/files/documents/amd-cdna-whitepaper.pdf

Matrix Cores in AMD Instinct™ MI200

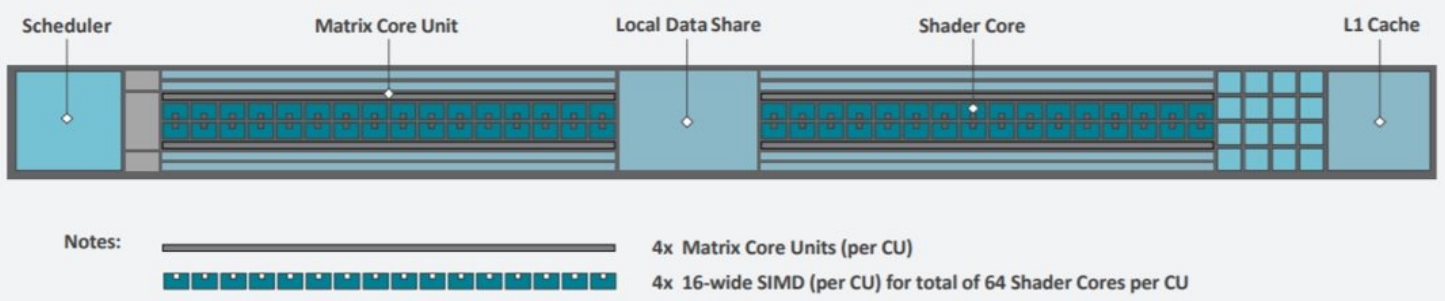


Figure 3 – Conceptional Block Diagram of an Enhanced Compute Unit (CU) with SIMD view of the AMD CDNA™ 2 architecture

- Matrix cores enhanced to perform FP64 operations in CDNA 2 architecture
- Doubled throughput for BF16

Data format	MI100 Flops/Clock/CU	MI250X Flops/Clock/CU
FP64	N/A	256
FP32	256	256
FP16	1024	1024
BF16	512	1024
INT8	1024	1024

https://www.amd.com/system/files/documents/amd-cdna2-white-paper.pdf

How to use AMD Matrix Cores?

- Mostly via rocBLAS or rocWMMA libraries (recommended)
- Writing kernels entirely in assembly (not practical)
- Sprinkling inline assembly in HIP kernels (not recommended)
- Using compiler intrinsics in HIP kernels (compiler is familiar with semantics but built-in functions may change in the future)

Syntax of MFMA Compiler Intrinsic

- Consider this MFMA operation: `D = AB + C` where A, B, C and D are all matrices

```
d = __builtin_amdgcn_mfma_CDFmt_MxNxKABFmt (a, b, c, cbsz, abid, blgp)
```

- `CDfmt` is the data format of the `C` & `D` matrices
- `ABfmt` is the data format of the `A` & `B` matrices
- `M`, `N` and `K` are matrix dimensions:
  - `mA[M][K]` Source `A` matrix
  - `mB[K][N]` Source `B` matrix
  - `mC[M][N]` Accumulation input matrix `C`
  - `mD[M][N]` Accumulation result matrix `D`
- `a` is the set of vector registers storing values from source matrix `A`
- `b` is the set of vector registers storing values from source matrix `B`
- `c` is the set of vector registers storing values from accumulation input matrix `C`
- `d` is the set of vector registers storing values of accumulation result matrix `D`

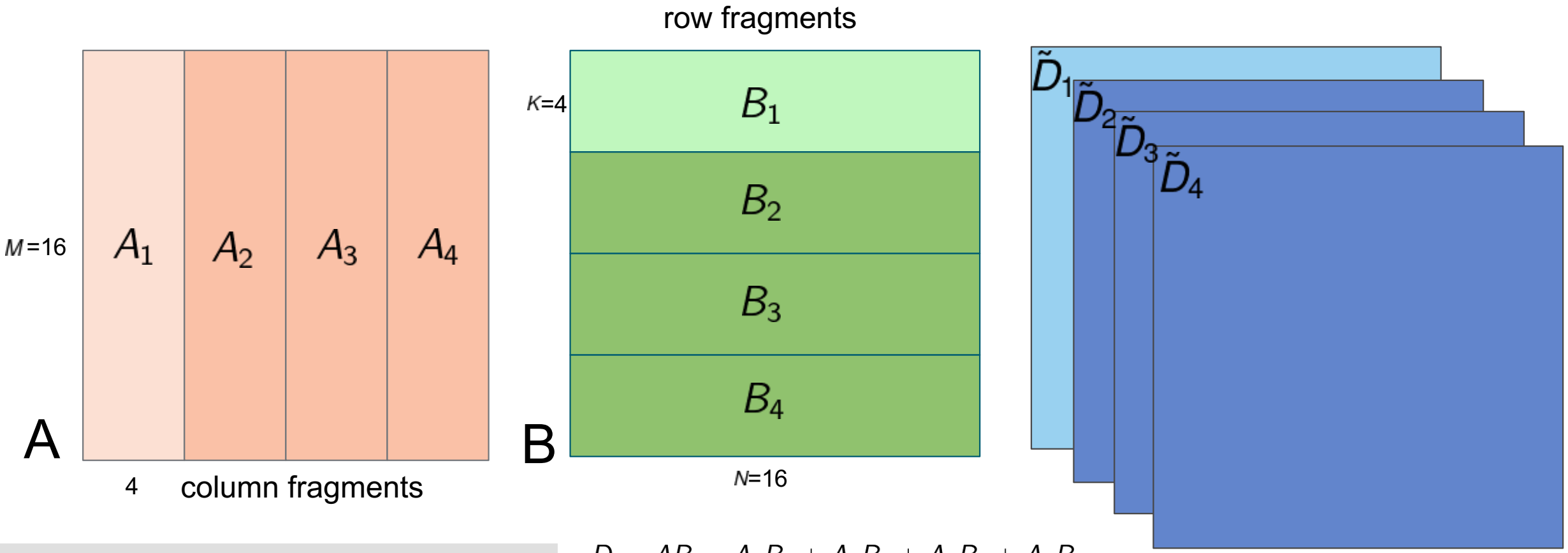
Wavefront-wide instructions!

Input and Output data are staged in lanes of vector registers used by the wavefront

https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-matrix-cores-readme/
AMD Instinct MI200 Instruction Set Architecture Reference Guide

AMD together we advance_

16x16x16 SGEMM Example of Multiplication with Outer Products



- Partition **A** into 16x4 "column fragments"
- Partition **B** into 4x16 "row fragments"
- Apply four 16x16x4 MFMA instructions to compute **AB**

$$D = AB = A_1B_1 + A_2B_2 + A_3B_3 + A_4B_4$$

$$D = \text{MFMA_}16x16x4(A_1, B_1, 0) \qquad D = A_1B_1$$

$$D = \text{MFMA_}16x16x4(A_2, B_2, D) \qquad D = A_1B_1 + A_2B_2$$

$$D = \text{MFMA_}16x16x4(A_3, B_3, D) \qquad \cdot$$

$$D = \text{MFMA_}16x16x4(A_4, B_4, D) \qquad \cdot$$

AMD Matrix Instruction Calculator

- https://github.com/RadeonOpenCompute/amd_matrix_instruction_calculator
- Generate information about MFMA instructions on AMD Radeon™ and AMD Instinct™ GPUs
  - Computational Throughput
  - Register Usage
- Understand the mapping between matrix elements and hardware registers
- Can also be used to understand WMMA instructions

AMD together we advance_

Example 1 – V_MFMA_F32_16x16x4F32 – Input layout

A

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,4
4,1	4,2	4,3	4,4
5,1	5,2	5,3	5,4
6,1	6,2	6,3	6,4
7,1	7,2	7,3	7,4
8,1	8,2	8,3	8,4
9,1	9,2	9,3	9,4
10,1	10,2	10,3	10,4
11,1	11,2	11,3	11,4
12,1	12,2	12,3	12,4
13,1	13,2	13,3	13,4
14,1	14,2	14,3	14,4
15,1	15,2	15,3	15,4
16,1	16,2	16,3	16,4

B

1,1	1,2	1,3	1,4	1,5	1,6	1,7	1,8	1,9	1,10	1,11	1,12	1,13	1,14	1,15	1,16
2,1	2,2	2,3	2,4	2,5	2,6	2,7	2,8	2,9	2,10	2,11	2,12	2,13	2,14	2,15	2,16
3,1	3,2	3,3	3,4	3,5	3,6	3,7	3,8	3,9	3,10	3,11	3,12	3,13	3,14	3,15	3,16
4,1	4,2	4,3	4,4	4,5	4,6	4,7	4,8	4,9	4,10	4,11	4,12	4,13	4,14	4,15	4,16

Matrix A has 16 x 4 elements
Matrix B has 4 x 16 elements
Assume Matrix C is zero

One invocation of
`__builtin_amdgcn_mfma_f32_16x16x4f32`

With a thread block of `(16,4)` threads i.e., 1 wavefront

Lane index	0	1	2		15	16	17	18		31	32	33	34		47	48	49	50		63
A layout	1,1	2,1	3,1	●●●	16,1	1,2	2,2	3,2	●●●	16,2	1,3	2,3	3,3	●●●	16,3	1,4	2,4	3,4	●●●	16,4
Lane index	0	1	2		15	16	17	18		31	32	33	34		47	48	49	50		63
B layout	1,1	1,2	1,3	●●●	1,16	2,1	2,2	2,3	●●●	2,16	3,1	3,2	3,3	●●●	3,16	4,1	4,2	4,3	●●●	4,16

2 vector registers hold values of A and B respectively (64 lanes in each register)

Example 1 – V_MFMA_F32_16x16x4F32 – Output layout

D

1,1	1,2	1,3	1,4	1,5	1,6	1,7	1,8	1,9	1,10	1,11	1,12	1,13	1,14	1,15	1,16
2,1	2,2	2,3	2,4	2,5	2,6	2,7	2,8	2,9	2,10	2,11	2,12	2,13	2,14	2,15	2,16
3,1	3,2	3,3	3,4	3,5	3,6	3,7	3,8	3,9	3,10	3,11	3,12	3,13	3,14	3,15	3,16
4,1	4,2	4,3	4,4	4,5	4,6	4,7	4,8	4,9	4,10	4,11	4,12	4,13	4,14	4,15	4,16
5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8	5,9	5,10	5,11	5,12	5,13	5,14	5,15	5,16
6,1	6,2	6,3	6,4	6,5	6,6	6,7	6,8	6,9	6,10	6,11	6,12	6,13	6,14	6,15	6,16
7,1	7,2	7,3	7,4	7,5	7,6	7,7	7,8	7,9	7,10	7,11	7,12	7,13	7,14	7,15	7,16
8,1	8,2	8,3	8,4	8,5	8,6	8,7	8,8	8,9	8,10	8,11	8,12	8,13	8,14	8,15	8,16
9,1	9,2	9,3	9,4	9,5	9,6	9,7	9,8	9,9	9,10	9,11	9,12	9,13	9,14	9,15	9,16
10,1	10,2	10,3	10,4	10,5	10,6	10,7	10,8	10,9	10,10	10,11	10,12	10,13	10,14	10,15	10,16
11,1	11,2	11,3	11,4	11,5	11,6	11,7	11,8	11,9	11,10	11,11	11,12	11,13	11,14	11,15	11,16
12,1	12,2	12,3	12,4	12,5	12,6	12,7	12,8	12,9	12,10	12,11	12,12	12,13	12,14	12,15	12,16
13,1	13,2	13,3	13,4	13,5	13,6	13,7	13,8	13,9	13,10	13,11	13,12	13,13	13,14	13,15	13,16
14,1	14,2	14,3	14,4	14,5	14,6	14,7	14,8	14,9	14,10	14,11	14,12	14,13	14,14	14,15	14,16
15,1	15,2	15,3	15,4	15,5	15,6	15,7	15,8	15,9	15,10	15,11	15,12	15,13	15,14	15,15	15,16
16,1	16,2	16,3	16,4	16,5	16,6	16,7	16,8	16,9	16,10	16,11	16,12	16,13	16,14	16,15	16,16

Matrix D has 16 x 16 elements

Lane index	0	1	2		15	16	17	18		31	32	33	34		47	48	49	50		63
D layout	1,1	1,2	1,3	●●●	1,16	5,1	5,2	5,3	●●●	5,16	9,1	9,2	9,3	●●●	9,16	13,1	13,2	13,3	●●●	13,16
	2,1	2,2	2,3	●●●	2,16	6,1	6,2	6,3	●●●	6,16	10,1	10,2	10,3	●●●	10,16	14,1	14,2	14,3	●●●	14,16
	3,1	3,2	3,3	●●●	3,16	7,1	7,2	7,3	●●●	7,16	11,1	11,2	11,3	●●●	11,16	15,1	15,2	15,3	●●●	15,16
	4,1	4,2	4,3	●●●	4,16	8,1	8,2	8,3	●●●	8,16	12,1	12,2	12,3	●●●	12,16	16,1	16,2	16,3	●●●	16,16

4 vector registers hold values of D

Summary

- AMD Matrix Cores provide hardware acceleration for Matrix Multiplication computations
  - CDNA and CDNA 2 architectures introduce new family of Matrix Fused Multiply Add (MFMA) instructions
- MFMA instructions are wavefront-wide instructions
- Available via rocBLAS and rocWMMA libraries
- Compiler Intrinsics can be used to leverage Matrix Cores
  - Requires careful mapping between input/output data and vector registers of wavefront

AMD
together we advance_

References

- AMD Matrix Cores lab notes: https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-matrix-cores-readme/
- AMD Instinct MI200 Instruction Set Architecture Reference Guide: https://developer.amd.com/wp-content/resources/CDNA2_Shader_ISA_18November2021.pdf
- AMD CDNA Architecture Whitepaper: https://www.amd.com/system/files/documents/amd-cdna-whitepaper.pdf
- AMD CDNA™ 2 Architecture Whitepaper: https://www.amd.com/system/files/documents/amd-cdna2-white-paper.pdf
- AMD Matrix Instruction Calculator Tool: https://github.com/RadeonOpenCompute/amd_matrix_instruction_calculator

Disclaimer

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED ‘AS IS.” AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AMD, the AMD Arrow logo, ROCm and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

The OpenMP name and the OpenMP logo are registered trademarks of the OpenMP Architecture Review Board.

HPE is a registered trademark of Hewlett Packard Enterprise Company and/or its affiliates.

Linux is the registered trademark of Linus Torvalds in the U.S. and other countries.

© 2023 Advanced Micro Devices, Inc. All rights reserved.

AMD